Exercício de alocação de recursos com cgroupv2

Objetivo: Aprender a configurar manualmente grupos de controle (cgroups) usando a versão 2 do sistema, limitando e distribuindo o uso de CPU entre processos, com alocação por peso (cpu.weight), limites (cpu.max) e pinagem de CPU (cpuset.cpus).

Parte 1 – Preparação do Ambiente

1. Suba uma VM usando a config de vagrant no anexo.

2. Torne-se root:
```
sudo su -
```

3. Verifique se o sistema está usando cgroups v2:
```
mount | grep cgroup2
```

4. Ative os controladores necessários:
```
echo "+cpu +cpuset" > /sys/fs/cgroup/cgroup.subtree_control
```

5. Crie dois cgroups:
```
mkdir /sys/fs/cgroup/labgrp1
mkdir /sys/fs/cgroup/labgrp2
```

6. Atribua ambos ao core 0:
```
echo 0 > /sys/fs/cgroup/labgrp1/cpuset.cpus
echo 0 > /sys/fs/cgroup/labgrp2/cpuset.cpus
```

7. Configure os pesos de CPU:
```
echo 256 > /sys/fs/cgroup/labgrp1/cpu.weight
echo 1024 > /sys/fs/cgroup/labgrp2/cpu.weight
```

8. Permita uso completo da CPU:
```
echo max > /sys/fs/cgroup/labgrp1/cpu.max
echo max > /sys/fs/cgroup/labgrp2/cpu.max
```

Parte 2 – Executar Processo no Grupo 1 (labgrp1)

Inicie uma sessão tmux:
```
tmux new -s grp1
```

Saia do tmux sem encerrá-lo: *Pressione Ctrl + B, depois D*

Encontrar o PID do bash:

1. Descubra o PID do processo tmux:
   ```
   ps -ef | grep tmux
   ```

   Exemplo de saída:
   ```
   root   13245   1   0   10:30   ?   00:00:00   tmux new -s grp1
   ```

2. Descubra o processo bash filho (executado pelo tmux na nova sessão) buscando pelo PID do tmux (o bash tera o PID do tmux como PPID):

   ```
   ps -ef | grep 13245
   ```

   Exemplo:

   ```
   root   13245   1       0   10:30   ?       00:00:00   tmux new -s grp1
   root   13247   13245   0   10:30   pts/1   00:00:00   bash
   ```

3. Atribua o processo ao cgroup:
   ```
   echo 13247 > /sys/fs/cgroup/labgrp1/cgroup.procs
   ```

4. Retorne à sessão:
   ```
   tmux attach -t grp1
   ```

5. Execute o gerador de carga:
   ```
   while :; do :; done
   ```

6. Saia do tmux novamente sem encerrar a sessão

Parte 3 – Executar Processo no Grupo 2 (labgrp2)

1. Abra outra sessão na máquina com outro usuário, que não o root. (exemplo: o usuário vagrant mesmo):

2. Inicie uma sessão tmux com esse usuário:
   ```
   tmux new -s grp2
   ```

3. Saia do tmux:
   Pressione Ctrl + B, depois D.

4. Descubra o PID do tmux desse usuário:
```
ps -ef | grep tmux
```

Exemplo de saída:
```
root     13245   1       0   10:30   ?       00:00:00   tmux new -s grp1
vagrant  14600   1       0   10:32   ?       00:00:00   tmux new -s grp2
```

5. Veja o bash filho:
```
ps -ef | grep 14600
```

Exemplo:
```
vagrant 14600     1     0  10:32   ?   00:00:00  tmux new -s grp2
vagrant 14602 14600     0  10:32 pts/2 00:00:00  bash
```

6. Como root, na outra sessão, mova o processo do bash para o cgroup:
```
echo 14602 > /sys/fs/cgroup/labgrp2/cgroup.procs
```

7. Nesta sessão como aluno2, volte para o tmux do usuário:
```
tmux attach -t grp2
```

8. Gere carga com o aplicativo stress:
```
stress --cpu 1
```

(Se não estiver instalado: `sudo apt install stress -y`)

Parte 4 – Verificação com top

1. Execute o comando:
```
top
```

2. Dentro do top, pressione:
s → digite 1 (para atualizar a cada 1 segundo)
1 → para mostrar o uso de CPU por core
P (maiúsculo) → para ordenar por uso de CPU

O que observar:

- O processo bash do grupo 1 estará identificado por “bash”, usando CPU de forma constante.
- O processo do grupo 2 será identificado como “stress”.
- Ambos estarão usando o core 0, conforme definido em cpuset.cpus.

Parte 5 – Ver estatísticas dos cgroups

Verifique o tempo de CPU acumulado:

```
cat /sys/fs/cgroup/labgrp1/cpu.stat
cat /sys/fs/cgroup/labgrp2/cpu.stat
```

Parte 6 - Questões a serem respondidas:

1) O que mostra o output dinâmico do top no uso de CPU pelos processos que foram criados? Quanto cada um está usando de CPU?
2) Qual a relação entre o uso de CPU real por cada grupo e as configurações feitas para cada grupo?
3) Explique os dados visualizados nas estatísticas.
4) Explique a sintaxe usada (o que significa cada opcao de CLI / argumento) dos comandos usados. Copie as linhas de commandos **inteiras** e explique cada um dos argumentos usados.
    a. stress
    b. tmux
    c. sudo

Caso 2: Parte 7 – Modificar cpu.max

Aplique os seguintes valores máximos de utilização de CPU para os grupos:

```
echo "60000 100000" > /sys/fs/cgroup/labgrp1/cpu.max
echo "60000 100000" > /sys/fs/cgroup/labgrp2/cpu.max
```

Refaça os testes e a análise solicitada nos passos 4 a 6, com o novo setup.

Considerações Finais

- No primeiro cenário (cpu.max = max), o peso (cpu.weight) define como a CPU será dividida: o grupo com peso 1024 usará mais CPU que o com peso 256.
- No segundo cenário (cpu.max = 60%), ambos os grupos são limitados de forma rígida e o peso causará um efeito diferente na utilização final de CPU.

Anexo: Configuração do vagrant para o exercício.

```
Vagrant.configure("2") do |config|
    config.vm.box = "bento/ubuntu-22.04"
    config.vm.hostname = "lab-cgroup"
    config.vm.provider "virtualbox" do |vb|
      vb.gui = true
      vb.memory = "1024"
      vb.cpus = 2
      vb.name = "lab-cgroup"
    end
 end
```